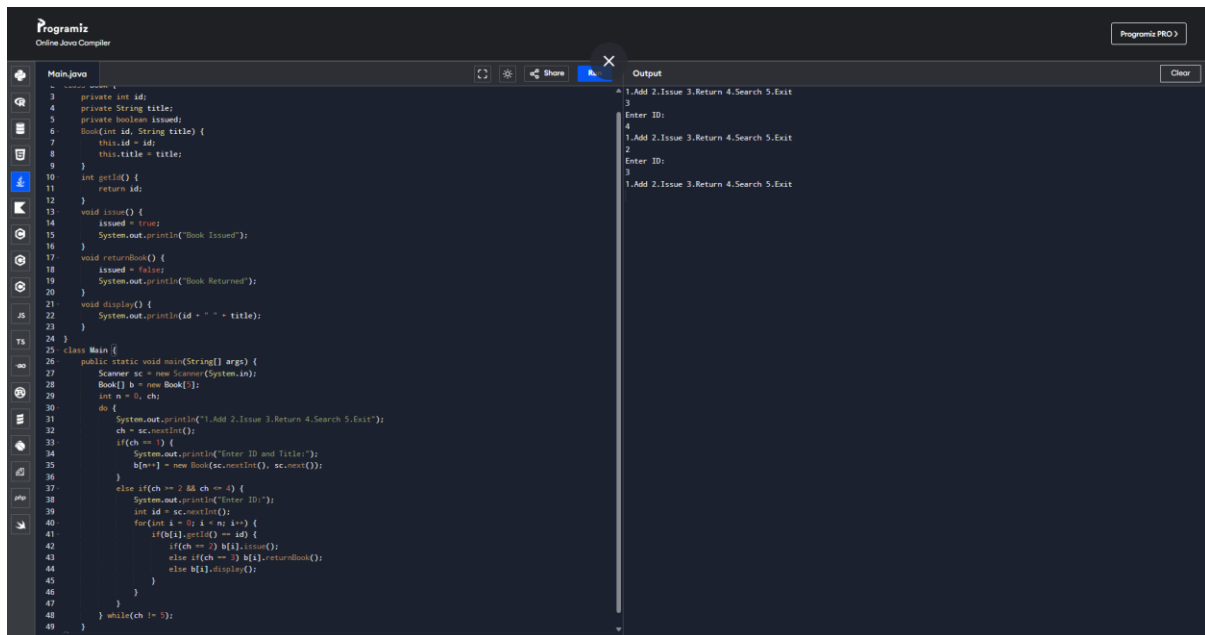


CSA0911

Programming in JAVA

A.H.N.V.S.Vamsi (192372242)-----CO1-AT1-----

1. Library management system



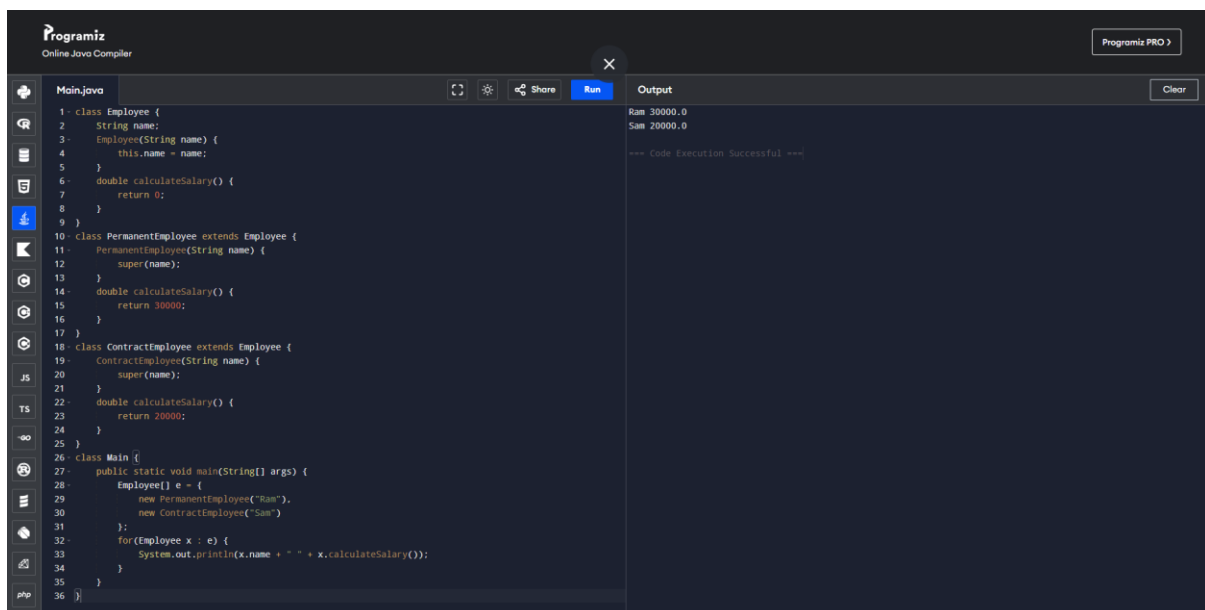
The screenshot shows the Programiz Online Java Compiler interface. The code in Main.java defines a Book class with attributes id, title, and issued, and methods for adding, returning, and displaying books. The Main class uses a Scanner to take user input and interacts with the Book class. The output shows the program running successfully with the following sequence of operations: 1.Add 2.Issue 3.Return 4.Search 5.Exit, followed by Enter ID: 1, 1.Add 2.Issue 3.Return 4.Search 5.Exit, Enter ID: 2, 1.Add 2.Issue 3.Return 4.Search 5.Exit, and Enter ID: 3.

```
3 private int id;
4 private String title;
5 private boolean issued;
6 Book(int id, String title) {
7     this.id = id;
8     this.title = title;
9 }
10 int getId() {
11     return id;
12 }
13 void issue() {
14     issued = true;
15     System.out.println("Book Issued");
16 }
17 void returnBook() {
18     issued = false;
19     System.out.println("Book Returned");
20 }
21 void display() {
22     System.out.println(id + " " + title);
23 }
24 }
25 class Main {
26     public static void main(String[] args) {
27         Scanner sc = new Scanner(System.in);
28         Book[] b = new Book[3];
29         int ch = 0, ch2;
30         do {
31             System.out.println("1.Add 2.Issue 3.Return 4.Search 5.Exit");
32             ch = sc.nextInt();
33             if(ch == 1) {
34                 System.out.println("Enter ID and Title:");
35                 b[ch2] = new Book(sc.nextInt(), sc.next());
36             }
37             else if(ch == 2 && ch2 < 4) {
38                 System.out.println("Enter ID:");
39                 int id = sc.nextInt();
40                 for(int i = 0; i < b.length; i++) {
41                     if(b[i].getId() == id) {
42                         if(ch2 == 2) b[i].issue();
43                         else if(ch2 == 3) b[i].returnBook();
44                         else b[i].display();
45                     }
46                 }
47             }
48             } while(ch != 5);
49     }
```

Output:

```
1.Add 2.Issue 3.Return 4.Search 5.Exit
3
Enter ID:
1
1.Add 2.Issue 3.Return 4.Search 5.Exit
2
Enter ID:
3
1.Add 2.Issue 3.Return 4.Search 5.Exit
```

2. Employee Payroll System



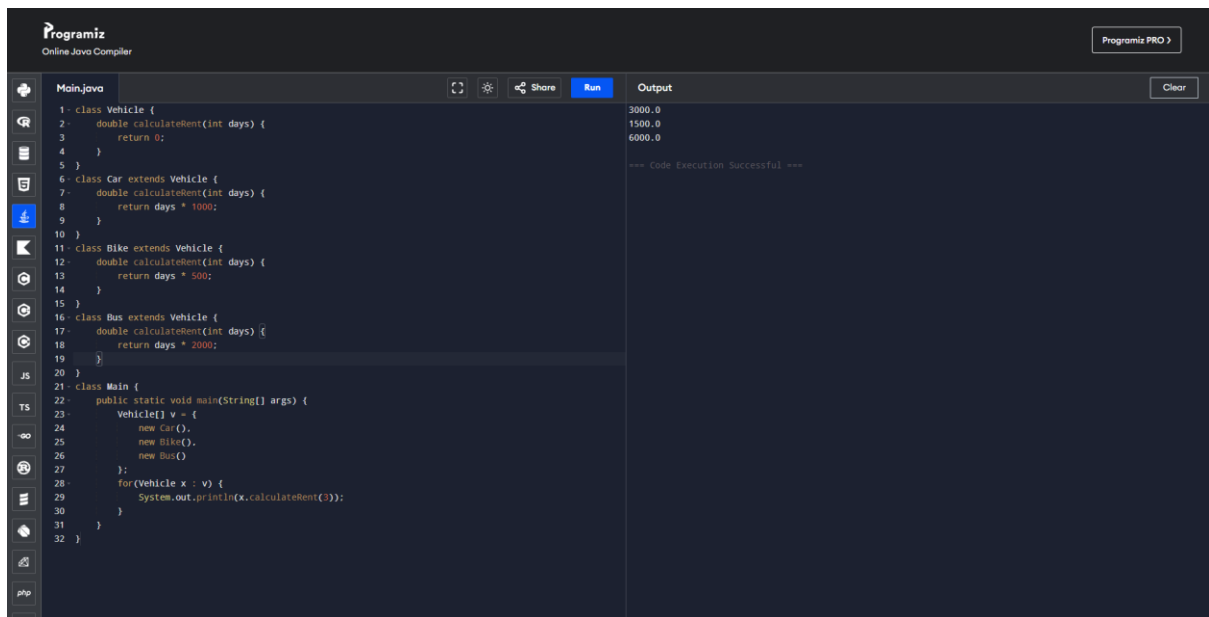
The screenshot shows the Programiz Online Java Compiler interface. The code in Main.java defines an Employee class with attributes name and salary, and methods for calculating salary. It also defines PermanentEmployee and ContractEmployee classes that inherit from Employee. The Main class uses a Scanner to take user input and interacts with the Employee class. The output shows the program running successfully with the following sequence of operations: Ram 30000.0, Sam 20000.0, followed by --- Code Execution Successful ---.

```
1 class Employee {
2     String name;
3     Employee(String name) {
4         this.name = name;
5     }
6     double calculateSalary() {
7         return 0;
8     }
9 }
10 class PermanentEmployee extends Employee {
11     PermanentEmployee(String name) {
12         super(name);
13     }
14     double calculateSalary() {
15         return 30000;
16     }
17 }
18 class ContractEmployee extends Employee {
19     ContractEmployee(String name) {
20         super(name);
21     }
22     double calculateSalary() {
23         return 20000;
24     }
25 }
26 class Main {
27     public static void main(String[] args) {
28         Employee[] e = {
29             new PermanentEmployee("Ram"),
30             new ContractEmployee("Sam")
31         };
32         for(Employee x : e) {
33             System.out.println(x.name + " " + x.calculateSalary());
34         }
35     }
36 }
```

Output:

```
Ram 30000.0
Sam 20000.0
--- Code Execution Successful ---
```

3. Vehicle Rental System



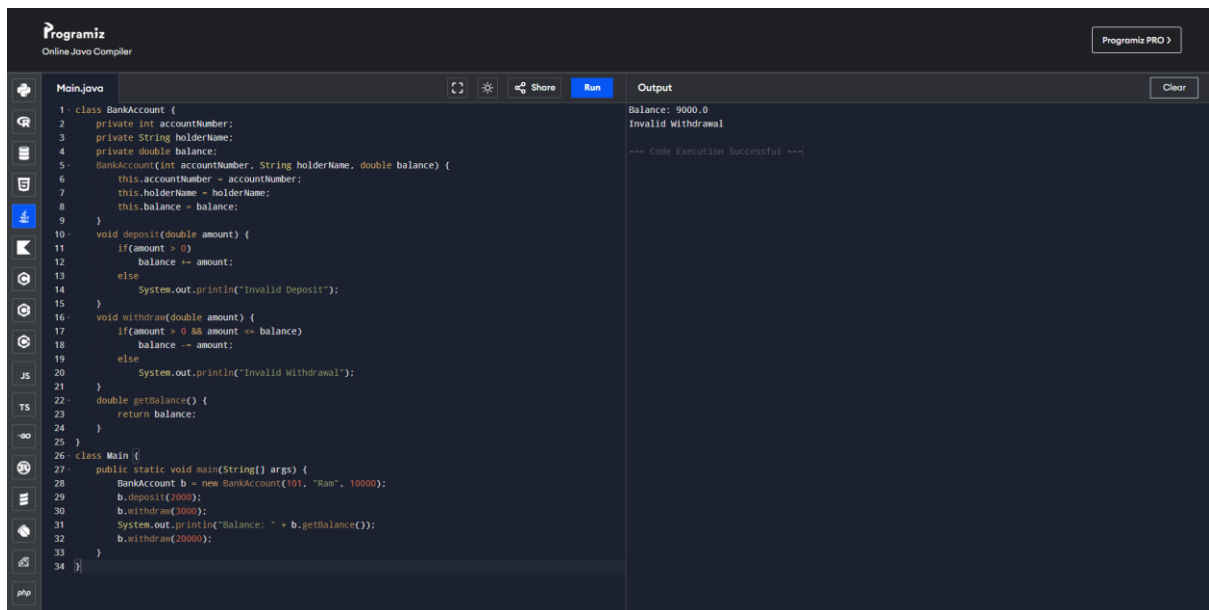
The screenshot shows the Programiz Online Java Compiler interface. The code is written in Java and defines a hierarchy of vehicle classes. The `Vehicle` class has a `calculateRent` method that returns 0. The `Car`, `Bike`, and `Bus` classes extend `Vehicle` and override the `calculateRent` method to return 1000, 500, and 2000 respectively. The `Main` class contains a `main` method that creates instances of `Car`, `Bike`, and `Bus`, and prints the result of `calculateRent` for each. The output shows the results of the calculations: 3000.0 for Car, 1500.0 for Bike, and 6000.0 for Bus. The code execution is successful.

```
1 class Vehicle {
2     double calculateRent(int days) {
3         return 0;
4     }
5 }
6 class Car extends Vehicle {
7     double calculateRent(int days) {
8         return days * 1000;
9     }
10 }
11 class Bike extends Vehicle {
12     double calculateRent(int days) {
13         return days * 500;
14     }
15 }
16 class Bus extends Vehicle {
17     double calculateRent(int days) {
18         return days * 2000;
19     }
20 }
21 class Main {
22     public static void main(String[] args) {
23         Vehicle[] v = {
24             new Car(),
25             new Bike(),
26             new Bus()
27         };
28         for(Vehicle x : v) {
29             System.out.println(x.calculateRent(3));
30         }
31     }
32 }
```

Output:

```
3000.0
1500.0
6000.0
--- Code Execution Successful ---
```

4. Bank Account System



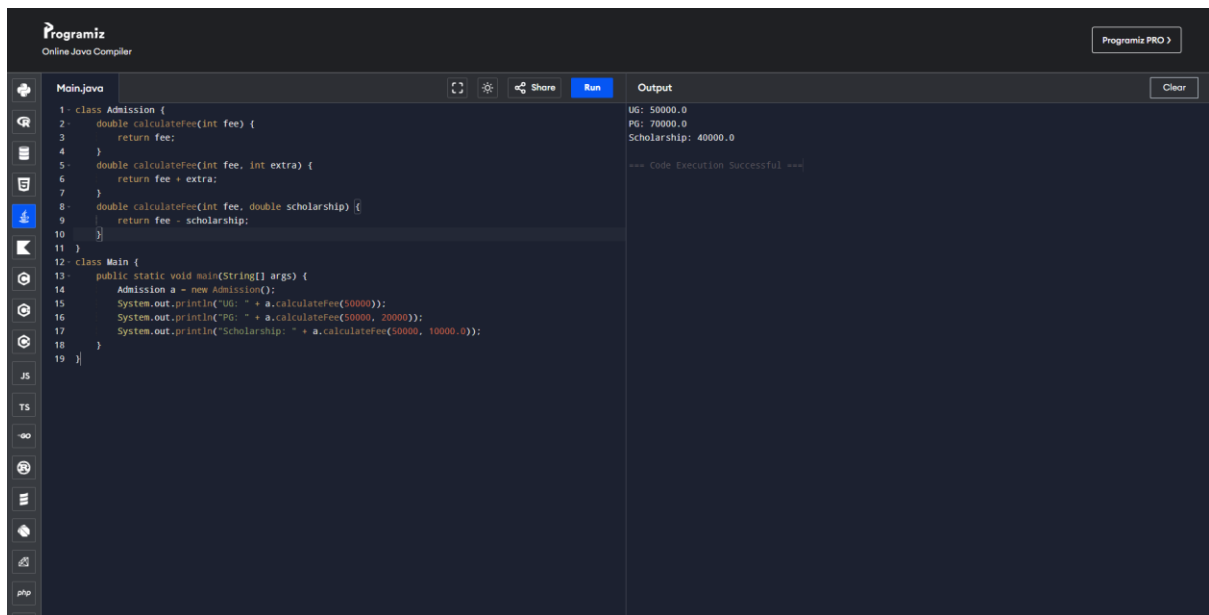
The screenshot shows the Programiz Online Java Compiler interface. The code is written in Java and defines a `BankAccount` class with private attributes `accountNumber` and `holderName`, and a public attribute `balance`. The `BankAccount` class has a constructor that takes `accountNumber`, `holderName`, and `balance` as parameters. It also has methods for `deposit`, `withdraw`, and `getBalance`. The `deposit` method checks if the amount is greater than 0 and adds it to the balance. The `withdraw` method checks if the amount is greater than 0 and less than or equal to the balance, and subtracts it from the balance. The `getBalance` method returns the current balance. The `Main` class contains a `main` method that creates a `BankAccount` object, deposits 2000, withdraws 3000, prints the balance, and withdraws 20000. The output shows the results of the operations: Balance: 9000.0, Invalid Withdrawal. The code execution is successful.

```
1 class BankAccount {
2     private int accountNumber;
3     private String holderName;
4     private double balance;
5     BankAccount(int accountNumber, String holderName, double balance) {
6         this.accountNumber = accountNumber;
7         this.holderName = holderName;
8         this.balance = balance;
9     }
10    void deposit(double amount) {
11        if(amount > 0)
12            balance += amount;
13        else
14            System.out.println("Invalid Deposit");
15    }
16    void withdraw(double amount) {
17        if(amount > 0 && amount <= balance)
18            balance -= amount;
19        else
20            System.out.println("Invalid Withdrawal");
21    }
22    double getBalance() {
23        return balance;
24    }
25 }
26 class Main {
27     public static void main(String[] args) {
28         BankAccount b = new BankAccount(101, "Ran", 10000);
29         b.deposit(2000);
30         b.withdraw(3000);
31         System.out.println("Balance: " + b.getBalance());
32         b.withdraw(20000);
33     }
34 }
```

Output:

```
Balance: 9000.0
Invalid Withdrawal
--- Code Execution Successful ---
```

5. University Admission System



The screenshot shows the Programiz Online Java Compiler interface. The code in Main.java defines an Admission class with methods to calculate fees and a Main class that tests these methods. The output shows the results of the calculations.

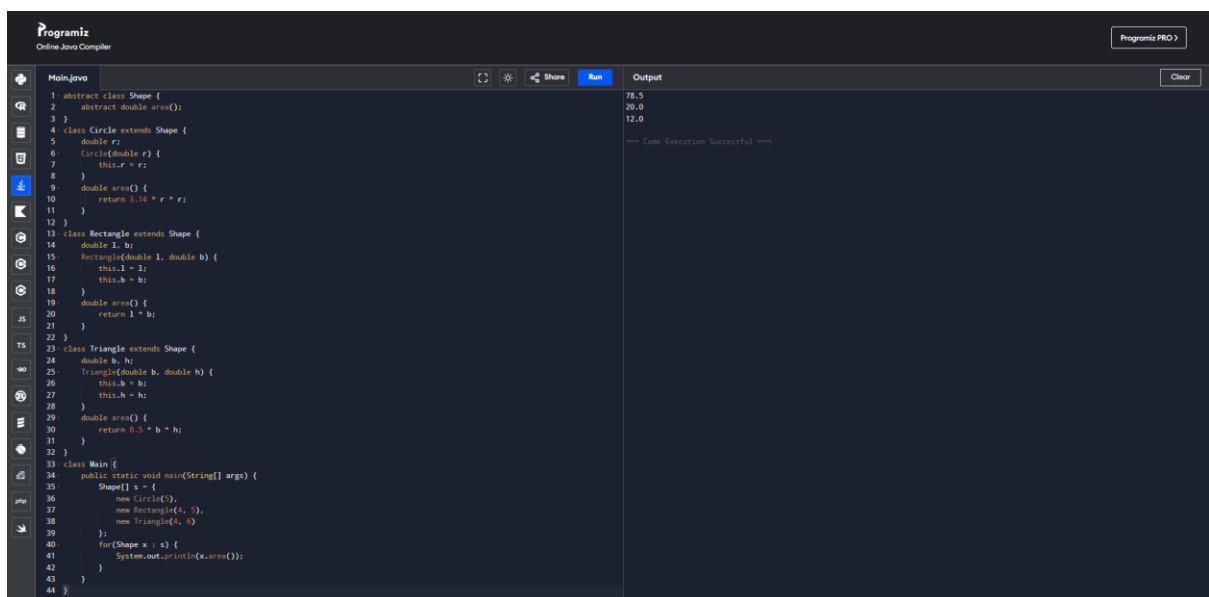
```
1 class Admission {
2     double calculateFee(int fee) {
3         return fee;
4     }
5     double calculateFee(int fee, int extra) {
6         return fee + extra;
7     }
8     double calculateFee(int fee, double scholarship) {
9         return fee - scholarship;
10    }
11 }
12 class Main {
13     public static void main(String[] args) {
14         Admission a = new Admission();
15         System.out.println("UG: " + a.calculateFee(50000));
16         System.out.println("PG: " + a.calculateFee(50000, 20000));
17         System.out.println("Scholarship: " + a.calculateFee(50000, 10000.0));
18     }
19 }
```

Output:

```
UG: 50000.0
PG: 70000.0
Scholarship: 40000.0

=== Code Execution Successful ===
```

6. Shape Area Calculator



The screenshot shows the Programiz Online Java Compiler interface. The code defines an abstract Shape class and its subclasses Circle, Rectangle, and Triangle, each with an area() method. The Main class creates instances of these shapes and prints their areas.

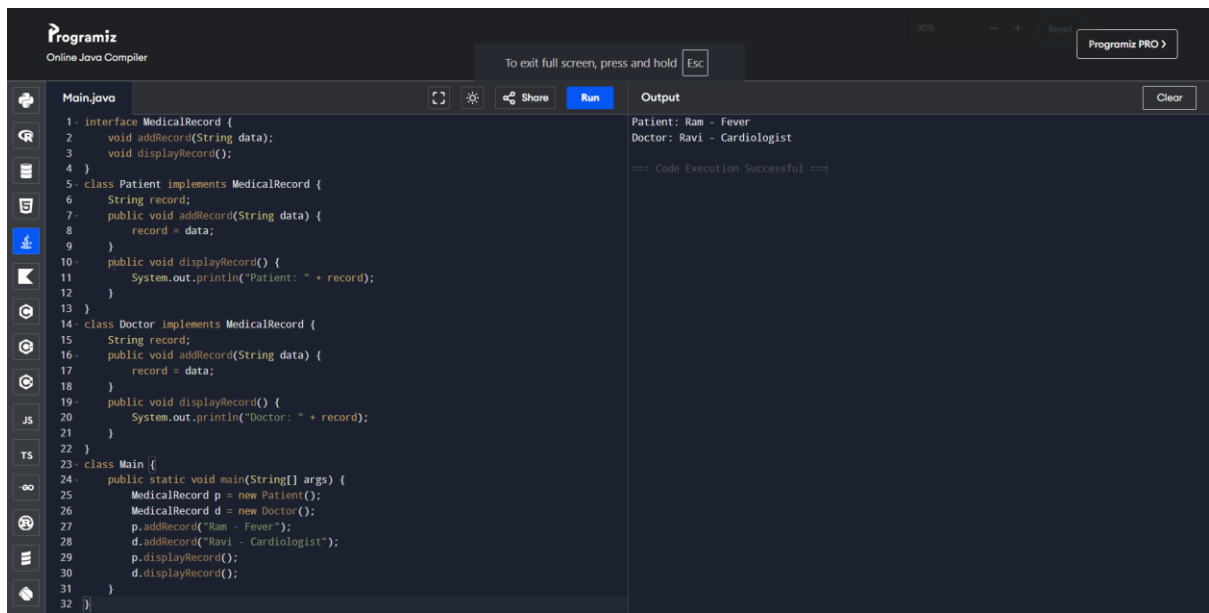
```
1 abstract class Shape {
2     abstract double area();
3 }
4 class Circle extends Shape {
5     double r;
6     Circle(double r) {
7         this.r = r;
8     }
9     double area() {
10        return 3.14 * r * r;
11    }
12 }
13 class Rectangle extends Shape {
14     double l, b;
15     Rectangle(double l, double b) {
16         this.l = l;
17         this.b = b;
18     }
19     double area() {
20        return l * b;
21    }
22 }
23 class Triangle extends Shape {
24     double b, h;
25     Triangle(double b, double h) {
26         this.b = b;
27         this.h = h;
28     }
29     double area() {
30        return 0.5 * b * h;
31    }
32 }
33 class Main {
34     public static void main(String[] args) {
35         Shape[] s = {
36             new Circle(5),
37             new Rectangle(4, 5),
38             new Triangle(4, 5)
39         };
40         for(Shape x : s) {
41             System.out.println(x.area());
42         }
43     }
44 }
```

Output:

```
78.5
20.0
12.0

=== Code Execution Successful ===
```

7. Hospital Management System



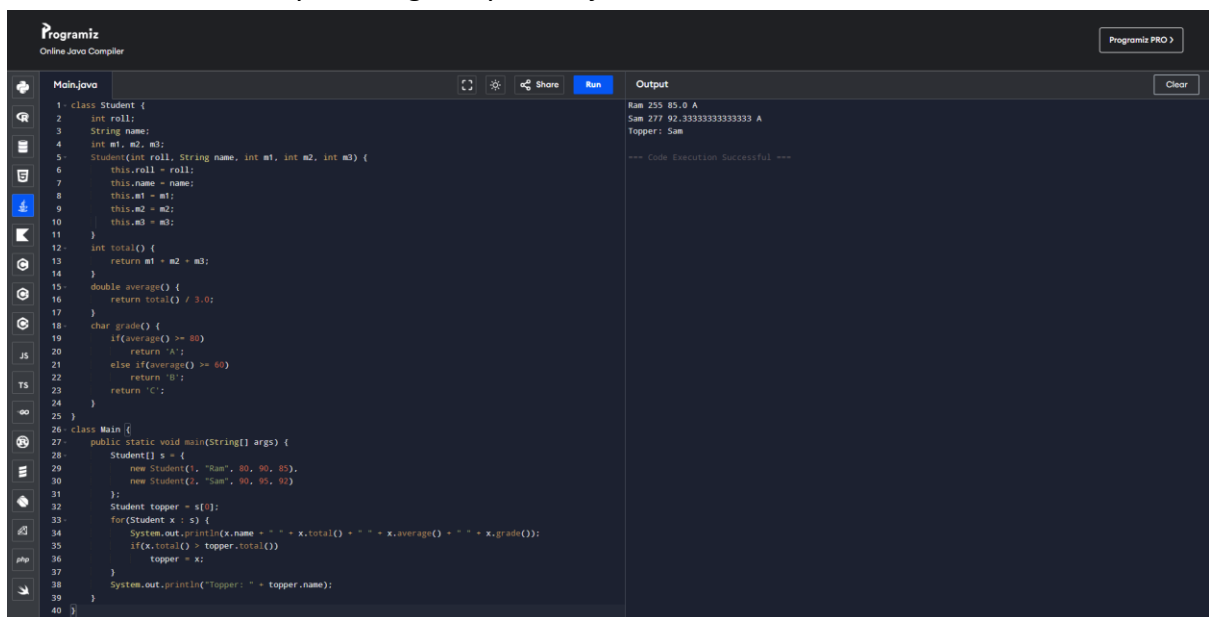
The screenshot shows the Programiz Online Java Compiler interface. The code defines an interface `MedicalRecord` with methods `addRecord(String data)` and `displayRecord()`. Two classes, `Patient` and `Doctor`, implement this interface. The `Main` class creates instances of `Patient` and `Doctor`, adds records, and displays them. The output shows the records for Patient (Ram - Fever) and Doctor (Ravi - Cardiologist).

```
1- interface MedicalRecord {
2-     void addRecord(String data);
3-     void displayRecord();
4- }
5- class Patient implements MedicalRecord {
6-     String record;
7-     public void addRecord(String data) {
8-         record = data;
9-     }
10-    public void displayRecord() {
11-        System.out.println("Patient: " + record);
12-    }
13- }
14- class Doctor implements MedicalRecord {
15-     String record;
16-     public void addRecord(String data) {
17-         record = data;
18-     }
19-     public void displayRecord() {
20-         System.out.println("Doctor: " + record);
21-     }
22- }
23- class Main {
24-     public static void main(String[] args) {
25-         MedicalRecord p = new Patient();
26-         MedicalRecord d = new Doctor();
27-         p.addRecord("Ram - Fever");
28-         d.addRecord("Ravi - Cardiologist");
29-         p.displayRecord();
30-         d.displayRecord();
31-     }
32- }
```

Output:

```
Patient: Ram - Fever
Doctor: Ravi - Cardiologist
=== Code Execution Successful ===
```

9. Student Result Analyzer using Arrays of Objects



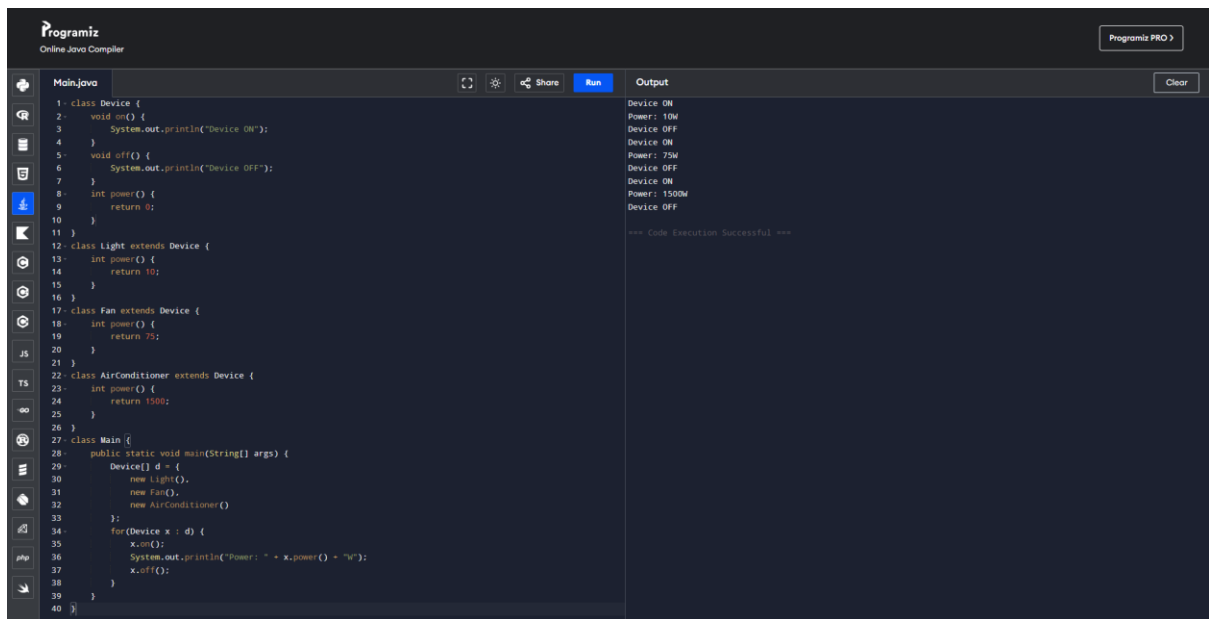
The screenshot shows the Programiz Online Java Compiler interface. The code defines a `Student` class with attributes `roll`, `name`, `m1`, `m2`, and `m3`. It includes methods for calculating total marks, average marks, and determining the grade. The `Main` class creates an array of `Student` objects, iterates through them to find the topper, and prints the results. The output shows the details for Ram and Sam, with Sam being the topper.

```
1- class Student {
2-     int roll;
3-     String name;
4-     int m1, m2, m3;
5-     Student(int roll, String name, int m1, int m2, int m3) {
6-         this.roll = roll;
7-         this.name = name;
8-         this.m1 = m1;
9-         this.m2 = m2;
10-        this.m3 = m3;
11-    }
12-    int total() {
13-        return m1 + m2 + m3;
14-    }
15-    double average() {
16-        return total() / 3.0;
17-    }
18-    char grade() {
19-        if(average() >= 80)
20-            return 'A';
21-        else if(average() >= 60)
22-            return 'B';
23-        return 'C';
24-    }
25- }
26- class Main {
27-     public static void main(String[] args) {
28-         Student[] s = {
29-             new Student(1, "Ram", 80, 90, 85),
30-             new Student(2, "Sam", 90, 95, 92)
31-         };
32-         Student topper = s[0];
33-         for(Student x : s) {
34-             System.out.println(x.name + " " + x.total() + " " + x.average() + " " + x.grade());
35-             if(x.total() > topper.total())
36-                 topper = x;
37-         }
38-         System.out.println("Topper: " + topper.name);
39-     }
40- }
```

Output:

```
Ram 255 85.0 A
Sam 277 92.33333333333333 A
Topper: Sam
=== Code Execution Successful ===
```

10. Smart Home Device Controller using Hierarchical Inheritance



The screenshot displays the Programiz Online Java Compiler interface. The left sidebar contains icons for various programming languages: Java, JavaScript, TypeScript, C++, C, Python, PHP, and others. The main editor area shows a Java file named 'Main.java' with the following code:

```
1 class Device {
2     void on() {
3         System.out.println("Device ON");
4     }
5     void off() {
6         System.out.println("Device OFF");
7     }
8     int power() {
9         return 0;
10    }
11 }
12 class Light extends Device {
13     int power() {
14         return 10;
15     }
16 }
17 class Fan extends Device {
18     int power() {
19         return 75;
20     }
21 }
22 class AirConditioner extends Device {
23     int power() {
24         return 1500;
25     }
26 }
27 class Main {
28     public static void main(String[] args) {
29         Device[] d = {
30             new Light(),
31             new Fan(),
32             new AirConditioner()
33         };
34         for(Device x : d) {
35             x.on();
36             System.out.println("Power: " + x.power() + "W");
37             x.off();
38         }
39     }
40 }
```

The right sidebar shows the 'Output' tab with the following text:

```
Device ON
Power: 10W
Device OFF
Device ON
Power: 75W
Device OFF
Device ON
Power: 1500W
Device OFF
=== Code Execution Successful ===
```